

A Fashions - Development Report & Interview Q&A

The build process, the questions it invites, terms answered, and sample code

Nirmalya Mandal - SAP Labs Interview Prep

Reconstructed from the 115-commit development history

Contents

How to use this report	3
Part 1 - The development story	4
Phase 1 - Foundation and the hero (mid-Oct 2025)	4
Phase 2 - Sections and motion	4
Phase 3 - Pages and content	4
Phase 4 - Security and SEO hardening	4
Phase 5 - Assets, optimisation, and deployment	4
Part 2 - Questions and detailed answers	5
“Why Next.js for an e-commerce site?”	5
“How did you get a 100 Lighthouse SEO score?”	5
“What is hCaptcha, and why did you add it?”	5
“How did you secure the contact form?”	5
“What is XSS / CSRF and how did you handle them?”	5
“What is lazy loading, and why ImageKit?”	5
“What does CMS-compatible mean here?”	5
“How did you deploy it?”	6
“What is a reverse proxy, and why put Nginx in front of Next.js?”	6
“How does the SSL automation work?”	6
“Why a VPS instead of managed hosting like Vercel?”	6
Part 3 - The bait-terms dictionary	7
Part 4 - Sample code snippets (pen and paper)	9
The hooks to drop on purpose	11

How to use this report

A Fashions (built at No Strategy Studios) is your production e-commerce project - a premium storefront for an international leather-goods manufacturer. It is your proof of **security hardening, SEO, and shipping a real public site**. Structure: **Part 1** build story, **Part 2** Q&A with answers, **Part 3** bait-terms dictionary, **Part 4** sample code. Say a term, pause, let them ask.

What it is in one breath: a polished, animation-rich e-commerce site for a leather brand, built end to end - a fast, responsive storefront, hardened against the bots and abuse any public site faces, and tuned to a perfect SEO score.

This is a genuine full-stack **and** DevOps story: you built the site and deployed it yourself on a **GoDaddy VPS** with Nginx and automated SSL - so the reverse-proxy and SSL work is truly yours to claim.

Part 1 - The development story

Phase 1 - Foundation and the hero (mid-Oct 2025)

- Set up the **Next.js** (TypeScript) project from a clean slate.
- Built the hero with a background **video** and a **curtain reveal** effect.
- Made the navbar responsive.

Phase 2 - Sections and motion

- Added **parallax scrolling**, **GSAP** animations, and **Lenis** smooth scrolling.
- Built the key-figures, WhoWeAre, LetsTalk, and footer sections; a **tilted gallery** and gradient treatments.
- Built the Our Story page and a custom 404 page.

Phase 3 - Pages and content

- Contact page with an **email-sending** feature and a pre-filled contact flow.
- Catalog page with all product imagery.
- Made every page **CMS-compatible** so content could be managed without code changes.
- Favicon and responsive fixes.

Phase 4 - Security and SEO hardening

- **Tightened the contact form security** and added **hCaptcha** to stop bots.
- Mitigated security issues flagged during review (headers, input handling).
- Implemented **SEO** (structured metadata) - the resume's 100 Lighthouse SEO score.
- Added **lazy loading** for images.

Phase 5 - Assets, optimisation, and deployment

- Moved images to **ImageKit** for optimised delivery.
- Added and later removed **Cloudflare** analytics.
- Upgraded to **Next.js 16** and tidied the file structure.
- Deployed to production on a **GoDaddy VPS**: a Linux server with an **Nginx reverse proxy** and **automated SSL** (Certbot). Vercel was used only for preview deployments during development.

Part 2 - Questions and detailed answers

“Why Next.js for an e-commerce site?”

For a shop, SEO and load speed are money - customers must find you on Google, and pages must feel instant. Next.js does **server-side rendering**, so search engines and users get full page content immediately, unlike plain React which ships a blank page that fills in later. That single difference is why a public storefront uses Next.js.

“How did you get a 100 Lighthouse SEO score?”

Structured metadata on every page, Schema.org structured data so search engines know exactly what each page is, a generated sitemap, semantic HTML, and image optimisation with lazy loading so the page is fast. Lighthouse rewards fast, well-described, accessible pages, so it is really the sum of those.

“What is hCaptcha, and why did you add it?”

hCaptcha is a challenge that tells humans and bots apart, similar to reCAPTCHA but more privacy-focused. I added it to the contact form because a public form is a magnet for spam bots. It stops automated submissions without adding much friction for real users.

“How did you secure the contact form?”

Layered: hCaptcha to block bots, server-side validation of every field, rate limiting so one source cannot flood it, and a honeypot field - an invisible input that humans never fill but naive bots do, which quietly filters them. And I never render user input back into the page unescaped, which closes XSS.

“What is XSS / CSRF and how did you handle them?”

XSS - Cross-Site Scripting - is an attacker injecting malicious JavaScript into a page so it runs in other users' browsers; I prevent it by escaping output and letting the framework sanitise, never injecting raw HTML. CSRF - Cross-Site Request Forgery - tricks a logged-in user's browser into an unwanted request; mitigated with same-site cookies and tokens on state-changing requests. On a mostly-static marketing site the bigger surface is the contact form, which is where I focused.

“What is lazy loading, and why ImageKit?”

Lazy loading means images load only as they are about to scroll into view, so the initial page is light and fast. ImageKit is a media CDN that serves optimised, correctly-sized, modern-format images from the edge - so a heavy, image-rich leather catalogue still loads quickly.

“What does CMS-compatible mean here?”

I structured the pages so their content comes from a data layer rather than being hardcoded, so the client could later manage copy and images through a CMS without touching code. It is the same separation-of-content-from-presentation idea.

“How did you deploy it?”

I deployed it to production on a **GoDaddy VPS** - a Linux server I configured myself. The Next.js app runs as a service on the box, and **Nginx** sits in front as a **reverse proxy**, receiving all web traffic on 80/443 and forwarding it to the app, while also terminating HTTPS. SSL is automated with **Certbot** (Let's Encrypt), so the certificates renew themselves. During development I used Vercel for quick preview deployments, but production is the VPS. This is one of two projects where I own the whole server, DokLink being the other.

“What is a reverse proxy, and why put Nginx in front of Next.js?”

The Next.js app listens on an internal port; I do not expose that directly. Nginx sits in front and forwards requests to it - that is the reverse proxy. It lets Nginx handle HTTPS termination, gzip, security headers, and static-file serving, and it means I can restart or swap the app behind it without changing what the outside world talks to. It is the standard way to run a Node app in production.

“How does the SSL automation work?”

Certbot obtains a free certificate from Let's Encrypt using the ACME protocol - it proves I control the domain, gets a certificate, and installs it into Nginx. A renewal timer runs periodically and renews before expiry, so HTTPS never lapses without me touching it.

“Why a VPS instead of managed hosting like Vercel?”

Managed hosting is easier but costs more and gives less control. A VPS gave me full control of the stack - the server, Nginx, SSL, the runtime - at a fraction of the cost, with the trade-off that I take on the operational responsibility. For a client site where I wanted control and low cost, that was the right call, and I was comfortable owning the ops because I do the same for DokLink.

Part 3 - The bait-terms dictionary

Next.js / server-side rendering

React framework that renders pages on the server for SEO and speed. *Say it:* “Next.js with SSR, because a shop lives or dies on search and load speed.”

SEO / Lighthouse / Schema.org / sitemap

Practices and a scoring tool for search visibility; structured data and sitemaps help crawlers. *Say it:* “100 Lighthouse SEO via metadata, Schema.org data, a sitemap, and fast images.”

hCaptcha

A privacy-focused bot-vs-human challenge. *Say it:* “hCaptcha on the contact form to stop spam bots.”

Rate limiting

Capping requests per source over time. *Say it:* “The form is rate-limited so one source can’t flood it.”

Honeypot

An invisible field bots fill and humans don’t. *Say it:* “A honeypot silently filters naive bots with zero user friction.”

XSS / CSRF

Script injection / forged cross-site requests. *Say it:* “XSS is blocked by escaping output; CSRF by same-site cookies and tokens.”

Lazy loading

Loading images only as they near the viewport. *Say it:* “Lazy loading keeps the first paint light on an image-heavy catalogue.”

ImageKit / image optimisation

A media CDN serving optimised images. *Say it:* “Images go through ImageKit - sized, modern formats, edge-delivered.”

GSAP / parallax / Lenis

Animation engine, depth-scroll effect, smooth-scroll library. *Say it:* “GSAP and Lenis drive the parallax and smooth scroll.”

CMS-compatible

Content sourced from data, ready for a CMS. *Say it:* “Every page is CMS-compatible - content is data, not hardcoded.”

GoDaddy VPS / Nginx reverse proxy / Certbot SSL

A self-managed Linux server; Nginx forwards traffic to the app and terminates HTTPS; Certbot auto-renews certificates. *Say it:* “Deployed on a GoDaddy VPS with Nginx as a reverse proxy and Certbot-automated SSL - I own the whole server.” (*Vercel was only for dev previews.*)

Part 4 - Sample code snippets (pen and paper)

SEO metadata + Schema.org structured data (Next.js)

```
export const metadata = {
  title: "A Fashions - Premium Leather",
  description: "Handcrafted leather goods.",
  openGraph: { images: ["/og.png"] },
};

// JSON-LD structured data in the page
<script type="application/ld+json"
  dangerouslySetInnerHTML={{ __html: JSON.stringify({
    "@context": "https://schema.org",
    "@type": "Organization",
    name: "A Fashions",
  }) }} />
```

Say aloud: “Metadata plus Schema.org JSON-LD so crawlers know exactly what the page is.”

Contact API route - hCaptcha + basic rate limit

```
const hits = new Map<string, number>();

export async function POST(req: Request) {
  const ip = req.headers.get("x-forwarded-for") ?? "unknown";
  hits.set(ip, (hits.get(ip) ?? 0) + 1);
  if ((hits.get(ip) ?? 0) > 5) return Response.json({ error: "rate" }, { status: 429 });

  const { token, name, email, honeypot } = await req.json();
  if (honeypot) return Response.json({ ok: true }); // bot trap
  const ok = await verifyHCaptcha(token); // human?
  if (!ok) return Response.json({ error: "captcha" }, { status: 400 });
  await sendEmail({ name, email });
  return Response.json({ ok: true });
}
```

Say aloud: “Rate limit per IP, a honeypot to catch bots, verify hCaptcha, then send - layered defence on a public form.”

Optimised, lazy-loaded image (Next.js)

```
import Image from "next/image";
```

```
<Image src={product.url} alt={product.name}  
  width={800} height={600} loading="lazy" />  // lazy + optimised
```

Say aloud: “next/image lazy-loads and serves optimised sizes, so an image-heavy catalogue stays fast.”

A scroll-triggered GSAP parallax

```
import gsap from "gsap";  
import { ScrollTrigger } from "gsap/ScrollTrigger";  
gsap.registerPlugin(ScrollTrigger);  
  
gsap.to(".bg", {  
  y: 200, // move slower than scroll = parallax  
  scrollTrigger: { trigger: ".section", scrub: true },  
});
```

Say aloud: “ScrollTrigger ties the background’s movement to scroll position - that’s the parallax.”

The hooks to drop on purpose

1. Next.js SSR for e-commerce SEO
2. 100 Lighthouse SEO (metadata, Schema.org, sitemap, fast images)
3. hCaptcha + honeypot + rate limiting on the contact form
4. XSS / CSRF handling
5. Lazy loading + ImageKit
6. GSAP parallax
7. **GoDaddy VPS deployment** - Nginx reverse proxy + Certbot SSL (a real DevOps hook, genuinely yours)

This project gives you a **second** VPS/Nginx/SSL story alongside DokLink - so you can speak to owning a Linux server with real confidence.